

Universidade Federal de Minas Gerais  
Departamento de Ciência da Computação  
Curso de Graduação em Ciência da Computação

**Trabalho Prático 2 - FSPD**  
**Armazenamento Chave-Valor Distribuído**

Lucas Affonso Pires  
Matrícula: 2023028420

Belo Horizonte, Novembro de 2025

# 1 Introdução

Este documento descreve as principais decisões de projeto tomadas durante a implementação do exercício de programação de um sistema chave-valor distribuído usando gRPC e Python. O foco é justificar as escolhas de arquitetura, estruturas de dados e lógica de comunicação. O projeto foi dividido em três partes, começando com um simples servidor de pares (chave-valor), evoluindo para um sistema com registro central (similar ao Napster) e, por fim, para uma rede hierárquica de super-pares. A ferramenta central utilizada foi o gRPC para comunicação por RPC.

## 2 Parte 1: Servidor de Pares (Chave-Valor)

### 2.1 Definição do Serviço (pares.proto)

A interface de serviço foi definida com quatro RPCs principais: `Insercao`, `Consulta`, `Ativacao` e `Termino`.

- **Mensagens:** Decidiu-se por mensagens simples e específicas para cada RPC (ex: `MsgInsercao`, `MsgValor`) para clareza, em vez de reutilizar mensagens genéricas.
- **Ativação:** A RPC `Ativacao` foi incluída desde o início, mesmo sem funcionalidade na Parte 1, para manter a interface do serviço estável ao longo das fases.

### 2.2 Implementação do Servidor (svc\_par.py)

- **Armazenamento:** Foi utilizado um dicionário Python (`dict`) para o armazenamento interno dos pares (chave, valor) devido à sua eficiência em buscas ( $\mathcal{O}(1)$  em média).
- **Concorrência:** Um `threading.Lock` foi adicionado para proteger o acesso ao dicionário. Embora o `ThreadPoolExecutor` do gRPC gerencie os threads, qualquer acesso a dados compartilhados (o dicionário) entre as chamadas deve ser serializado para evitar *race conditions*.
- **Término (Termino RPC):** A decisão mais crítica foi como parar o servidor sem interromper a resposta da RPC. A solução foi usar `threading.Thread` para invocar `server.stop(0)` em um thread separado, com um pequeno *delay* (0.1s). Isso permite que o servicer retorne a mensagem de sucesso ao cliente antes que o processo do servidor seja efetivamente encerrado.

## 3 Parte 2: Servidor Concentrador

### 3.1 Definição do Serviço (central.proto)

O serviço central introduziu três novas RPCs: `Registro`, `Busca` e `Pareamento`.

- **Listas de Chaves:** Para a RPC `Registro`, a palavra-chave `repeated int32` foi usada para permitir que o servidor de pares enviasse sua lista completa de chaves de uma só vez, o que é mais eficiente do que múltiplas chamadas.

### 3.2 Implementação do Servidor (svc\_cen.py)

- **Armazenamento:** O servidor central mantém um dicionário `key_map` que mapeia `chave (int)` -> `localizador (str)`.
- **Colisões:** Conforme especificado, o tratamento de colisões é uma simples sobrescrita. O último servidor a registrar uma chave "ganha".

### 3.3 Atualização do Servidor de Pares (Ativação)

Nesta fase, o servidor de pares (na RPC `Ativacao`) torna-se, ele mesmo, um cliente gRPC.

- **Cliente-Servidor:** Ele cria um canal para o servidor central (cujo endereço foi recebido via argumento de linha de comando) e invoca a RPC `Registro`.
- **Identificação:** Para se identificar, o servidor de pares usa `socket.getfqdn()` combinado com seu porto para criar seu próprio localizador de serviço (ex: "maquina.dcc.ufmg.br:5555").

### 3.4 Cliente Central (cln\_cen.py)

- **RPC Dupla:** O comando 'B' (Busca) no cliente central implementa a lógica "Napster". Ele primeiro faz uma RPC `Busca` ao servidor central. Se receber um localizador válido, ele imediatamente abre um novo canal gRPC para esse localizador e faz uma segunda RPC (`Consulta`) ao servidor de pares para obter o valor.

## 4 Parte 3: Rede Hierárquica (Super-Pares)

### 4.1 Lógica de Pareamento

O servidor central agora pode atuar como super-par (ativado pelo segundo argumento de linha de comando).

- **Lista de Pares:** Foi adicionada uma lista (`self.peers`) ao `CentralServicer` para armazenar os localizadores de outros super-pares conhecidos, preenchida pela RPC `Pareamento`.

### 4.2 Implementação da Busca (Alagamento)

Esta foi a mudança mais complexa, implementando um alagamento (flooding) controlado.

- **Ordem de Busca:** Ao receber uma RPC `Busca`, o super-par primeiro consulta seu próprio mapa local (`self.key_map`).
- **Falha Local:** Se a chave não for encontrada localmente, o servidor itera sobre sua lista `self.peers` e encaminha a consulta `Busca` para cada um deles.
- **Retorno Rápido:** Assim que qualquer um dos pares retorna um localizador válido, a busca é interrompida e o resultado é retornado imediatamente ao solicitante.

### 4.3 Prevenção de Loops

O alagamento pode criar loops infinitos em redes com ciclos.

- **Parâmetro quem\_pedi:** A mensagem `MsgBusca` foi modificada para incluir um campo `string quem_pedi`.
- **Lógica de Encaminhamento:** Ao encaminhar uma busca, o servidor se identifica neste campo. Ao receber uma busca, o servidor verifica se o solicitante está em sua lista de pares e, o mais importante, **\*\*não\*\*** encaminha a busca de volta para o par que acabou de perguntar. Isso quebra ciclos de tamanho 2 e, como a rede é uma árvore (conforme enunciado), garante o término.

## 5 Conclusão

A arquitetura baseada em gRPC permitiu uma evolução clara do sistema, desde um serviço monolítico simples até uma rede distribuída hierárquica. As decisões de projeto mais importantes envolveram o gerenciamento de estado concorrente (uso de `Lock`) e a implementação da lógica de roteamento de consultas (a `RPC Busca` da Parte 3), que efetivamente transforma cada servidor em um roteador de busca.